Assignment_2

Keerthi Priya Jakku

2026-02-18

Necessary packages

```
library(ggplot2)
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```
library(lattice)
library (caret)
library(class)

set.seed(123)
```

Load Data

```
UniversalBank<-read.csv("~/Desktop/R documents/Assignment files/FML_Assignment2/UniversalBank.csv")
```

Converting Education to Dummy variable

```
UniversalBank$Education <- as.factor(UniversalBank$Education)

dummy_model <- dummyVars(~ Education, data = UniversalBank)
dummy_data <- predict(dummy_model, newdata = UniversalBank)

UniversalBank <- UniversalBank[, !colnames(UniversalBank) %in% "Education"]
UniversalBank <- cbind(UniversalBank, dummy_data)
```

Excluding ID and Zip Code

```
UniversalBank<- UniversalBank[, !(names(UniversalBank) %in% c("ID", "ZIP.Code"))]
```

Converting Personal Loan to factor

```
UniversalBank$Personal.Loan <- as.factor(UniversalBank$Personal.Loan)
```

Renaming the column names

```
colnames(UniversalBank)[colnames(UniversalBank) == "EducationEducation_1"] <- "Education_1"
colnames(UniversalBank)[colnames(UniversalBank) == "EducationEducation_2"]<- "Education_2"
```

```
colnames(UniversalBank)[colnames(UniversalBank) == "EducationEducation_3"]<- "Education_3"
```

Partitioning the data into training (60%) and valida on (40%) sets

```
indextrain<-createDataPartition(UniversalBank$Personal.Loan,
                                p=0.6,
                                list=FALSE)
training<-UniversalBank[indextrain, ]
validation<- UniversalBank[-indextrain, ]
```

Seperating predictors and target

```
trainingX <- training[, !colnames(training) %in% "Personal.Loan"]
trainingY <- training$Personal.Loan

validationX <- validation[, !colnames(validation) %in% "Personal.Loan"]
validationY <- validation$Personal.Loan
```

Convert predictors to numeric

```
trainingX <- data.frame(lapply(trainingX, as.numeric))
validationX <- data.frame(lapply(validationX, as.numeric))
```

Standardizing the data for k-NN classification

```
prepro <- preProcess(trainingX, method = c("center", "scale"))

trainingX_scaled <- predict(prepro, trainingX)
validationX_scaled <- predict(prepro, validationX)
```

K-NN Classification Using K = 1

```
Predicted_KNN1 <- knn(train = trainingX_scaled,
                      test = validationX_scaled,
                      cl = trainingY,
                      k = 1)

confusionMatrix(Predicted_KNN1, validationY, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1793   56
##          1   15  136
##
##                Accuracy : 0.9645
##                  95% CI : (0.9554, 0.9722)
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7739
##
##  Mcnemar's Test P-Value : 2.063e-06
##
##             Sensitivity : 0.7083
##             Specificity : 0.9917
##          Pos Pred Value : 0.9007
```

```
##          Neg Pred Value : 0.9697
##              Prevalence : 0.0960
##          Detection Rate : 0.0680
##    Detection Prevalence : 0.0755
##       Balanced Accuracy : 0.8500
##
##        'Positive' Class : 1
##
```

Finding the best K

```
accuracy <- c()

for(i in 1:20){
  pred <- knn(train = trainingX_scaled,
              test = validationX_scaled,
              cl = trainingY,
              k = i)

  accuracy[i] <- mean(pred == validationY)
}

accuracy_table <- data.frame(
  k = 1:10,
  Accuracy = round(accuracy, 6)
)

print(accuracy_table)
```

```
##     k Accuracy
## 1   1   0.9645
## 2   2   0.9585
## 3   3   0.9635
## 4   4   0.9625
## 5   5   0.9595
## 6   6   0.9620
## 7   7   0.9585
## 8   8   0.9565
## 9   9   0.9535
## 10 10   0.9520
## 11  1   0.9515
## 12  2   0.9485
## 13  3   0.9495
## 14  4   0.9470
## 15  5   0.9470
## 16  6   0.9465
## 17  7   0.9445
## 18  8   0.9445
## 19  9   0.9440
## 20 10   0.9435
```

```
accuracy
```

```
##  [1] 0.9645 0.9585 0.9635 0.9625 0.9595 0.9620 0.9585 0.9565 0.9535 0.9520
## [11] 0.9515 0.9485 0.9495 0.9470 0.9470 0.9465 0.9445 0.9445 0.9440 0.9435
```

```
best_k <- which.max(accuracy)
best_k
```

```
## [1] 1
```

New customer data

```
new_customer <- data.frame(
  Age = 40,
  Experience = 10,
  Income = 84,
  Family = 2,
  CCAvg = 2,
  Mortgage = 0,
  Securities.Account = 0,
  CD.Account = 0,
  Online = 1,
  CreditCard = 1,
  Education.1 = 0,
  Education.2 = 1,
  Education.3 = 0
)

new_customer_scaled <- predict(prepro, new_customer)

knnpredict <- knn(train = trainingX_scaled,
              test = new_customer_scaled,
              cl= trainingY,
              k = best_k)

knnpredict
```

```
## [1] 0
## Levels: 0 1
```

Outcome: New Customer will not accept the Personal Loan

K-NN Classification Using k = 2

```
Predicted_KNN2 <- knn(train = trainingX_scaled,
                      test = validationX_scaled,
                      cl = trainingY,
                      k = 2)

conf_matrix_k2 <- confusionMatrix(Predicted_KNN2, validationY, positive = "1")
conf_matrix_k2
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1787   60
##          1   21  132
##
##                Accuracy : 0.9595
##                  95% CI : (0.9499, 0.9677)
##     No Information Rate : 0.904
```

```
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7434
##
##  Mcnemar's Test P-Value : 2.419e-05
##
##             Sensitivity : 0.6875
##             Specificity : 0.9884
##          Pos Pred Value : 0.8627
##          Neg Pred Value : 0.9675
##              Prevalence : 0.0960
##          Detection Rate : 0.0660
##    Detection Prevalence : 0.0765
##       Balanced Accuracy : 0.8379
##
##        'Positive' Class : 1
##
```

```
prediction2 <- knn(trainingX_scaled,
                   new_customer_scaled,
                   cl = trainingY,
                   k = 2)
```

Confusion Matrix Using k=2

```
knn_k2 <- knn(train = trainingX_scaled,
              test = validationX_scaled,
              cl = trainingY,
              k = 2)

conf_matrix_k2 <- confusionMatrix(knn_k2, validationY, positive = "1")
conf_matrix_k2
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1791   61
##          1   17  131
##
##                Accuracy : 0.961
##                  95% CI : (0.9516, 0.9691)
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 0.7497
##
##  Mcnemar's Test P-Value : 1.123e-06
##
##             Sensitivity : 0.6823
##             Specificity : 0.9906
##          Pos Pred Value : 0.8851
##          Neg Pred Value : 0.9671
##              Prevalence : 0.0960
##          Detection Rate : 0.0655
##    Detection Prevalence : 0.0740
```

```
##       Balanced Accuracy : 0.8364
##
##        'Positive' Class : 1
##
```

Dividing the data (50% training / 30% validation / 20% training)

```
indextrain2 <- createDataPartition(UniversalBank$Personal.Loan,
                                   p = 0.5,
                                   list = FALSE)

train2 <- UniversalBank[indextrain2, ]
temp <- UniversalBank[-indextrain2, ]

index_valid2 <- createDataPartition(temp$Personal.Loan,
                                   p = 0.6,
                                   list = FALSE)

valid2 <- temp[index_valid2, ]
test2 <- temp[-index_valid2, ]
```

Seperate predictors and target

```
train2X <- data.frame(lapply(train2[, !colnames(train2) %in% "Personal.Loan"], as.numeric))
train2Y <- train2$Personal.Loan

valid2X <- data.frame(lapply(valid2[, !colnames(valid2) %in% "Personal.Loan"], as.numeric))
valid2Y <- valid2$Personal.Loan

test2X <- data.frame(lapply(test2[, !colnames(test2) %in% "Personal.Loan"], as.numeric))
test2Y <- test2$Personal.Loan
```

Scale predictors

```
prepro2 <- preProcess(train2X, method = c("center", "scale"))
train2X_scaled <- predict(prepro2, train2X)
valid2X_scaled <- predict(prepro2, valid2X)
test2X_scaled <- predict(prepro2, test2X)
```

k-nn predictions

```
train2_pred <- knn(train2X_scaled, train2X_scaled, train2Y, k = best_k)
valid2_pred <- knn(train2X_scaled, valid2X_scaled, train2Y, k = best_k)
test2_pred <- knn(train2X_scaled, test2X_scaled, train2Y, k = best_k)
```

Confusion matrices

```
confusionMatrix(train2_pred, train2Y, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##            Reference
## Prediction    0    1
##          0 2260    0
##          1    0  240
##
##                Accuracy : 1
##                  95% CI : (0.9985, 1)
```

```
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : < 2.2e-16
##
##                   Kappa : 1
##
##  Mcnemar's Test P-Value : NA
##
##             Sensitivity : 1.000
##             Specificity : 1.000
##          Pos Pred Value : 1.000
##          Neg Pred Value : 1.000
##              Prevalence : 0.096
##          Detection Rate : 0.096
##    Detection Prevalence : 0.096
##       Balanced Accuracy : 1.000
##
##        'Positive' Class : 1
##
```

```
confusionMatrix(valid2_pred, valid2Y, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##          0 1335   49
##          1   21   95
##
##                Accuracy : 0.9533
##                  95% CI : (0.9414, 0.9634)
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : 7.606e-13
##
##                   Kappa : 0.7055
##
##  Mcnemar's Test P-Value : 0.00125
##
##             Sensitivity : 0.65972
##             Specificity : 0.98451
##          Pos Pred Value : 0.81897
##          Neg Pred Value : 0.96460
##              Prevalence : 0.09600
##          Detection Rate : 0.06333
##    Detection Prevalence : 0.07733
##       Balanced Accuracy : 0.82212
##
##        'Positive' Class : 1
##
```

```
confusionMatrix(test2_pred, test2Y, positive = "1")
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction   0   1
```

```
##          0 886  27
##          1  18  69
##
##                Accuracy : 0.955
##                  95% CI : (0.9402, 0.967)
##     No Information Rate : 0.904
##     P-Value [Acc > NIR] : 1.194e-09
##
##                   Kappa : 0.7294
##
##  Mcnemar's Test P-Value : 0.233
##
##             Sensitivity : 0.7188
##             Specificity : 0.9801
##          Pos Pred Value : 0.7931
##          Neg Pred Value : 0.9704
##              Prevalence : 0.0960
##          Detection Rate : 0.0690
##    Detection Prevalence : 0.0870
##       Balanced Accuracy : 0.8494
##
##        'Positive' Class : 1
##
```

Analysis: Thes dataset was split into 60% training and 40% validation sets after converting the Education variable into dummy variables and removing ID and ZIP code. When k = 1 was used, the model achieved a validation accuracy of about 96.45%, and the new customer was predicted as 0, meaning they are unlikely to accept the loan. After testing different k values from 1 to 20, k = 1 gave the highest accuracy and was chosen as the best value. However, k = 2 was also tested to reduce possible overfitting. Although its accuracy was slightly lower (around 95–96%), the performance was still good and the new customer was again classified as 0. When the data was repartitioned into 50% training, 30% validation, and 20% test sets, the training accuracy was very high while the validation and test accuracies were both around 95%. This suggests that k = 1 may slightly overfit the training data, while k = 2 provides a more stable model. Results also differ due to the randomness process. Overall, the k-NN model performs well in predicting loan acceptance, and the customer is predicted not to accept the loan.